Contents

ByteFloats.jl benchmark report

Generated: 2026-07-19 18:52 UTC · Julia 1.12.6 · arrowlake-s (24 logical CPUs, 4 Julia threads) · Float128 paths: enabled · Chairmarks 1.3.1

Reference format for per-operation tables: `Binary8p4se` under `(NearestTiesToEven, SatNone)`. Every table names its operand class: the scalar-operation tables appear in four variants — all code points (NaN and ±Inf sampled), NaN excluded, finite-only, and per-operation in-domain — and every other sampled table uses the all-code-points pool, identified in its note. Times are per call; medians with minima alongside. Methodology per the recorded benchmark doctrine: type-parameterized barriers, untimed setup, specialization preflight.

Core primitives

The decode/compare/step/classify layer plus the projection engine. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs
`decode`	1.4 ns	1.4 ns	0
`order_key`	1.5 ns	1.4 ns	0
`x < y`	1.4 ns	1.3 ns	0
`TotalOrder`	1.8 ns	1.7 ns	0
`Class`	1.7 ns	1.4 ns	0
`NextGreaterThan`	1.7 ns	1.3 ns	0
`project (RNE·SatNone)`	6.2 ns	1.7 ns	0
`project (StochasticA[8], R drawn)`	6.9 ns	1.7 ns	0

Scalar operations — unary (30) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
Abs	6.6 ns	4.4 ns	0
Negate	7.3 ns	4.5 ns	0
Recip	19.4 ns	7.9 ns	0
Sqrt	20.4 ns	4.9 ns	0
RSqrt	22.5 ns	10.8 ns	0
Sin	25.2 ns	5.1 ns	0
Tanh	26.9 ns	5.1 ns	0
ArcSin	27.0 ns	5.1 ns	0
Exp	27.1 ns	7.2 ns	0
ExpMinusOne	27.1 ns	5.3 ns	0
Exp2	27.2 ns	7.4 ns	0
ArcCos	27.6 ns	5.2 ns	0
ArcSinPi	27.8 ns	5.2 ns	0
Log	28.3 ns	5.3 ns	0
Cosh	28.9 ns	7.2 ns	0
LogOnePlus	29.3 ns	5.3 ns	0
Sinh	29.7 ns	5.4 ns	0
ArcTan	29.8 ns	5.3 ns	0
CosPi	30.0 ns	6.4 ns	0
ArcCosPi	30.2 ns	5.1 ns	0
TanPi	32.1 ns	5.1 ns	0
ArcSinh	35.5 ns	5.1 ns	0
ArcCosh	36.1 ns	5.0 ns	0
ArcTanh	37.7 ns	5.4 ns	0
ArcTanPi	41.5 ns	6.2 ns	0
Cos	44.4 ns	11.9 ns	0
Log2	51.3 ns	9.0 ns	0
Tan	56.6 ns	10.2 ns	0
SinPi	58.9 ns	9.8 ns	0
Softplus	72.5 ns	27.0 ns	0

Scalar operations — unary (30) — no NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	median	min	allocs
ArcCosh	1.6 ns	1.5 ns	0
RSqrt	1.7 ns	1.5 ns	0
ArcSinPi	2.2 ns	1.5 ns	0
Log	2.7 ns	1.5 ns	0
Log2	3.0 ns	1.5 ns	0
Sqrt	5.1 ns	1.8 ns	0

operation	median	min	allocs
ArcSin	9.2 ns	2.4 ns	0
Abs	11.0 ns	7.3 ns	0
Negate	11.8 ns	9.3 ns	0
ArcCosPi	14.2 ns	2.7 ns	0
TanPi	14.9 ns	4.9 ns	0
Recip	19.8 ns	1.5 ns	0
Exp	26.3 ns	6.7 ns	0
Tanh	26.8 ns	5.0 ns	0
Exp2	27.6 ns	6.8 ns	0
ArcTanh	29.2 ns	1.5 ns	0
CosPi	29.3 ns	6.1 ns	0
Cosh	29.6 ns	7.1 ns	0
SinPi	30.5 ns	4.9 ns	0
ExpMinusOne	34.0 ns	5.5 ns	0
LogOnePlus	34.9 ns	1.7 ns	0
ArcSinh	35.3 ns	4.9 ns	0
ArcCos	38.7 ns	3.3 ns	0
Softplus	40.2 ns	15.0 ns	0
Sin	47.5 ns	9.9 ns	0
Cos	47.7 ns	13.6 ns	0
ArcTanPi	51.9 ns	8.6 ns	0
ArcTan	56.6 ns	9.5 ns	0
Sinh	56.7 ns	9.9 ns	0
Tan	62.2 ns	9.2 ns	0

Scalar operations — unary (30) — no NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	median	min	allocs
ArcCosh	1.5 ns	1.5 ns	0
ArcTanh	1.6 ns	1.5 ns	0
Sqrt	1.8 ns	1.5 ns	0
RSqrt	1.9 ns	1.5 ns	0
Log2	2.1 ns	1.4 ns	0
ArcSin	2.6 ns	1.9 ns	0
Log	3.1 ns	1.5 ns	0
ArcCos	3.5 ns	1.7 ns	0
Abs	6.9 ns	4.4 ns	0
ArcCosPi	7.1 ns	1.4 ns	0
Negate	10.8 ns	5.3 ns	0
Sin	23.7 ns	1.5 ns	0
Cos	24.5 ns	1.5 ns	0
ArcSinPi	26.0 ns	1.5 ns	0
Recip	26.4 ns	2.0 ns	0
Exp2	26.7 ns	4.7 ns	0
LogOnePlus	28.3 ns	1.5 ns	0
Cosh	28.6 ns	4.8 ns	0
ExpMinusOne	28.6 ns	4.6 ns	0
Tanh	29.3 ns	4.9 ns	0

operation	median	min	allocs
CosPi	29.4 ns	1.5 ns	0
SinPi	30.5 ns	1.5 ns	0
Sinh	30.7 ns	4.9 ns	0
TanPi	32.3 ns	1.5 ns	0
Tan	33.0 ns	1.5 ns	0
ArcSinh	35.3 ns	4.7 ns	0
ArcTanPi	36.6 ns	5.2 ns	0
Softplus	37.6 ns	4.6 ns	0
Exp	39.6 ns	6.7 ns	0
ArcTan	47.5 ns	8.1 ns	0

Scalar operations — unary (30)

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median. Transcendental rows mix special-row fast returns with enclosure-path evaluations, so these are *scalar-path* costs; bulk unary work routes through 256-byte tables (see Array kernels).

operation	median	min	allocs
ArcCosh	1.6 ns	1.5 ns	0
Sqrt	2.4 ns	1.7 ns	0
ArcSinPi	2.6 ns	1.5 ns	0
RSqrt	3.7 ns	2.1 ns	0
Log	4.1 ns	1.4 ns	0
Log2	4.5 ns	1.5 ns	0
ArcSin	5.0 ns	1.7 ns	0
ArcTanh	5.0 ns	1.5 ns	0
Abs	7.0 ns	1.8 ns	0
Negate	9.9 ns	2.0 ns	0
TanPi	12.9 ns	1.5 ns	0
Sin	23.6 ns	1.5 ns	0
Cos	24.0 ns	1.5 ns	0
ArcCos	25.4 ns	1.7 ns	0
Tanh	26.6 ns	1.7 ns	0
Exp2	26.6 ns	1.7 ns	0
ExpMinusOne	26.9 ns	1.7 ns	0
ArcCosPi	27.3 ns	1.4 ns	0
LogOnePlus	27.4 ns	1.5 ns	0
Cosh	27.5 ns	1.8 ns	0
ArcTan	28.9 ns	1.8 ns	0
Sinh	29.1 ns	1.7 ns	0
CosPi	29.4 ns	1.5 ns	0
Tan	30.3 ns	1.6 ns	0
SinPi	30.4 ns	1.5 ns	0
ArcTanPi	31.2 ns	1.7 ns	0
ArcSinh	35.6 ns	1.7 ns	0
Softplus	37.9 ns	1.7 ns	0
Recip	38.3 ns	3.1 ns	0
Exp	50.2 ns	3.3 ns	0

Scalar operations — binary (18) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
MaximumMagnitude	6.8 ns	6.5 ns	0
MinimumMagnitude	6.8 ns	4.7 ns	0
MaximumMagnitudeNumber	6.8 ns	6.5 ns	0
MinimumMagnitudeNumber	6.9 ns	4.7 ns	0
Multiply	7.1 ns	4.7 ns	0
CopySign	7.2 ns	5.0 ns	0
MaximumFinite	7.7 ns	4.7 ns	0
MinimumFinite	7.7 ns	4.7 ns	0
Minimum	7.7 ns	6.3 ns	0
Add	7.8 ns	5.3 ns	0
Maximum	7.9 ns	4.7 ns	0
MinimumNumber	8.0 ns	4.7 ns	0
MaximumNumber	8.1 ns	4.8 ns	0
Subtract	10.6 ns	6.0 ns	0
Divide	24.9 ns	6.6 ns	0
Hypot	30.2 ns	7.5 ns	0
ArcTan2	32.5 ns	6.9 ns	0
ArcTan2Pi	34.5 ns	6.2 ns	0

Scalar operations — binary (18) — no NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	median	min	allocs
MinimumMagnitude	6.8 ns	4.6 ns	0
MaximumMagnitudeNumber	6.8 ns	6.4 ns	0
MaximumMagnitude	6.8 ns	6.4 ns	0
MinimumMagnitudeNumber	6.9 ns	4.8 ns	0
CopySign	7.1 ns	4.7 ns	0
Minimum	7.7 ns	4.7 ns	0
Maximum	7.8 ns	4.7 ns	0
MaximumNumber	7.8 ns	5.7 ns	0
MaximumFinite	7.8 ns	5.1 ns	0
Add	7.9 ns	5.7 ns	0
MinimumNumber	7.9 ns	6.3 ns	0
MinimumFinite	8.7 ns	5.1 ns	0
Multiply	10.3 ns	6.6 ns	0
Subtract	11.2 ns	6.2 ns	0
Divide	25.6 ns	2.5 ns	0
Hypot	29.4 ns	7.1 ns	0
ArcTan2	32.5 ns	6.3 ns	0
ArcTan2Pi	34.5 ns	6.4 ns	0

Scalar operations — binary (18) — no NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	median	min	allocs
MinimumMagnitude	6.8 ns	4.6 ns	0
MaximumMagnitudeNumber	7.0 ns	4.7 ns	0
Multiply	7.2 ns	4.7 ns	0
MaximumMagnitude	7.2 ns	5.1 ns	0
MinimumMagnitudeNumber	7.4 ns	4.7 ns	0
Maximum	7.7 ns	4.5 ns	0
Minimum	8.0 ns	4.8 ns	0
MaximumNumber	8.2 ns	4.7 ns	0
MinimumNumber	8.2 ns	4.9 ns	0
CopySign	9.1 ns	5.6 ns	0
MaximumFinite	10.2 ns	5.9 ns	0
Subtract	11.0 ns	3.0 ns	0
MinimumFinite	12.1 ns	7.3 ns	0
Add	12.2 ns	3.7 ns	0
Divide	22.2 ns	2.0 ns	0
Hypot	30.4 ns	5.2 ns	0
ArcTan2	32.9 ns	6.1 ns	0
ArcTan2Pi	34.6 ns	6.3 ns	0

Scalar operations — binary (18)

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	median	min	allocs
MinimumMagnitude	6.8 ns	1.5 ns	0
MaximumMagnitudeNumber	6.8 ns	4.7 ns	0
MinimumMagnitudeNumber	7.0 ns	4.8 ns	0
MaximumMagnitude	7.0 ns	1.8 ns	0
Multiply	7.1 ns	1.5 ns	0
MaximumFinite	7.6 ns	4.7 ns	0
Minimum	7.7 ns	1.5 ns	0
Maximum	7.7 ns	1.5 ns	0
MinimumFinite	7.8 ns	4.9 ns	0
MinimumNumber	7.8 ns	4.6 ns	0
MaximumNumber	7.9 ns	4.5 ns	0
CopySign	8.1 ns	1.6 ns	0
Add	9.7 ns	2.0 ns	0
Subtract	10.6 ns	2.8 ns	0
Divide	25.4 ns	2.4 ns	0
ArcTan2	32.0 ns	2.8 ns	0
Hypot	32.4 ns	2.0 ns	0
ArcTan2Pi	34.4 ns	2.8 ns	0

Scalar operations — ternary (3) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
`Clamp`	9.0 ns	5.0 ns	0
`FMA`	9.6 ns	7.2 ns	0
`FAA`	9.8 ns	8.1 ns	0

Scalar operations — ternary (3) — no NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	median	min	allocs
`Clamp`	9.2 ns	4.7 ns	0
`FMA`	9.7 ns	7.2 ns	0
`FAA`	9.8 ns	7.8 ns	0

Scalar operations — ternary (3) — no NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	median	min	allocs
`Clamp`	9.0 ns	4.9 ns	0
`FAA`	9.8 ns	6.5 ns	0
`FMA`	9.8 ns	6.6 ns	0

Scalar operations — ternary (3)

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	median	min	allocs
`Clamp`	8.9 ns	1.5 ns	0
`FMA`	9.5 ns	3.2 ns	0
`FAA`	9.5 ns	2.6 ns	0

Format sensitivity

Same three binary ops across formats; `Binary8p1uf` exercises the wide-exponent-spread escalations, small-K formats the tiny-table regime. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs
Add⟨Binary8p4se⟩	8.0 ns	1.9 ns	0
Divide⟨Binary8p4se⟩	18.8 ns	1.9 ns	0
Multiply⟨Binary8p4se⟩	7.3 ns	1.5 ns	0
Add⟨Binary8p3sf⟩	10.3 ns	2.4 ns	0
Divide⟨Binary8p3sf⟩	17.0 ns	1.9 ns	0
Multiply⟨Binary8p3sf⟩	6.9 ns	1.5 ns	0
Add⟨Binary8p1uf⟩	190.5 ns	0.0 ns	0
Divide⟨Binary8p1uf⟩	8.1 ns	1.9 ns	0
Multiply⟨Binary8p1uf⟩	6.6 ns	1.5 ns	0
Add⟨Binary5p2se⟩	7.9 ns	1.9 ns	0
Divide⟨Binary5p2se⟩	9.0 ns	1.9 ns	0
Multiply⟨Binary5p2se⟩	7.2 ns	1.5 ns	0
Add⟨Binary3p1se⟩	5.7 ns	1.9 ns	0
Divide⟨Binary3p1se⟩	5.5 ns	1.9 ns	0
Multiply⟨Binary3p1se⟩	5.6 ns	1.5 ns	0

Projection by rounding/saturation mode

`project(Binary8p4se, ρ, x)` over the mode vocabulary (stochastic budgets N = 8). Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs
NearestTiesToEven	6.9 ns	1.7 ns	0
NearestTiesToAway	14.1 ns	4.1 ns	0
TowardPositive	7.7 ns	2.2 ns	0
TowardNegative	5.6 ns	2.1 ns	0
TowardZero	5.0 ns	2.1 ns	0
ToOdd	6.3 ns	1.7 ns	0
StochasticA[8]	6.9 ns	1.7 ns	0
StochasticB[8]	8.1 ns	1.7 ns	0
StochasticC[8]	11.0 ns	2.4 ns	0
RNE · SatFinite	9.1 ns	2.3 ns	0
RNE · SatPropagate	6.3 ns	1.7 ns	0

Array kernels (vmap)

Warm caches: table specializations prebuilt, so table rows measure the gather; scalar-loop rows measure the full compute pipeline per element. The ternary row here is `Binary8p4se` (K=8, always the compute path); see the next section for how the ternary bitwidth policy behaves across K. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs	per element
`vmap unary (table gather), n=65536`	10.6 µs	9.0 µs	0	0.16 ns/elem — 6.18 Gelem/s

operation	median	min	allocs	per element
`vmap binary (table gather), n=65536`	21.6 µs	17.2 µs	0	0.33 ns/elem — 3.03 Gelem/s
`vmap ternary (scalar loop), n=65536`	285.9 µs	263.5 µs	22	4.36 ns/elem — 0.23 Gelem/s
`vmap binary stochastic (scalar loop), n=65536`	658.9 µs	567.9 µs	0	10.05 ns/elem — 0.1 Gelem/s
`vmap unary through PackedVector, n=65536`	85.2 µs	72.2 µs	519	1.3 ns/elem — 0.77 Gelem/s

Ternary bitwidth tiers (FMA/FAA)

`FMA/FAA/Clamp` are total functions on `2^(K1+K2+K3)` code points, but that count spans 512 B (K=3) to 16 MiB (K=8), so the array kernel tables small operand formats eagerly, tables mid-size ones adaptively (after enough elements amortize the build; not shown here — see the adaptive-cache gate in `test/ternary_opt.jl`), and always runs the scalar compute kernel at K=8, threaded above a size cutoff when `Threads.nthreads() > 1`. Each tier's optimized row is paired with a scalar-loop baseline (policy Refs forced off around the measurement, restored after) so the win is visible per tier; this process has 4 Julia threads. Same reference format, ρ, and operand pool discipline as Array kernels above.

operation	median	min	allocs	per element
`FMA K=4 (eager table), n=65536`	19.9 µs	19.0 µs	0	0.3 ns/elem — 3.29 Gelem/s
`FMA K=4 (eager table), scalar-loop baseline, n=65536`	1.07 ms	995.0 µs	0	16.34 ns/elem — 0.06 Gelem/s
`FMA K=6 (eager table), n=65536`	59.5 µs	52.6 µs	0	0.91 ns/elem — 1.1 Gelem/s
`FMA K=6 (eager table), scalar-loop baseline, n=65536`	1.27 ms	1.05 ms	0	19.4 ns/elem — 0.05 Gelem/s
`FMA K=8 (compute), n=65536`	278.7 µs	260.5 µs	22	4.25 ns/elem — 0.24 Gelem/s

operation	median	min	allocs	per element
FMA K=8 (compute), scalar-loop baseline, n=65536	1.01 ms	971.8 µs	0	15.45 ns/elem — 0.06 Gelem/s
FMA K=8 (compute), threaded [4t], n=65536	280.6 µs	261.8 µs	22	4.28 ns/elem — 0.23 Gelem/s
FMA K=8 (compute), sequential [1t], n=65536	990.1 µs	956.7 µs	0	15.11 ns/elem — 0.07 Gelem/s

Sorting (64 K values)

Counting sort is installed as the default algorithm for `Binary` vectors. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs
sort! (counting sort via defalg), n=65536	244.65 µs	201.5 µs	2
sort! (stock comparison sort), n=65536	3.6 ms	2.4 ms	3
sort! rev=true (counting sort), n=65536	333.3 µs	319.7 µs	2

Table builds (oracle + projection, Float128-first)

Cold cache per sample (`empty_tables!` in untimed setup); JIT pre-warmed. The warm-hit column is the steady-state cost of `get_table` when the specialization is already cached (median / min). Table entries enumerate every code point by construction (NaN and ±Inf included).

operation	median	min	allocs	warm hit
Exp⟨8p4se⟩ (256 entries)	34.8 µs	27.5 µs	257	124.2 ns / 66.8 ns
Tanh⟨8p4se⟩ (256 entries)	36.8 µs	27.7 µs	257	68.4 ns / 65.7 ns
Add⟨8p4se×8p4se⟩ (64 K entries)	22.89 ms	20.64 ms	785668	69.7 ns / 67.2 ns
Divide⟨8p4se×8p4se⟩ (64 K)	22.65 ms	20.51 ms	784906	70.0 ns / 67.3 ns
Add⟨8p1uf×8p1uf⟩ (64 K, wide-spread)	67.44 ms	61.74 ms	1916682	91.9 ns / 67.6 ns

Block and scaled operations

Elements `Binary8p4se`, scales `Binary8p1uf`, B = 32. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs	per lane
`BlockAdd (B=32)`	1.57 µs	1.46 µs	35	49.18 ns/lane
`BlockDotProduct → 8p4se (B=32)`	387.5 ns	300.0 ns	3	12.11 ns/lane
`BlockReduceAdd → 8p4se (B=32)`	1.59 µs	42.1 ns	0	49.67 ns/lane
`ConvertToBlockMaxAbsF (B=32)`	3.1 µs	2.76 µs	75	96.88 ns/lane

Conversions and packed storage

Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs	per element
`T(::UInt8) code-point constructor`	1.8 ns	1.3 ns	0	
`rawvalue (unchecked kernel route)`	1.4 ns	1.3 ns	0	
`T(::Float64) numeric constructor (projects)`	6.3 ns	1.7 ns	0	
`Convert 8p4se → 8p3se (scalar)`	6.9 ns	1.7 ns	0	
`Float64 → 8p4se (project)`	6.2 ns	1.7 ns	0	
`PackedVector pack, n=65536`	36.3 µs	33.8 µs	3	0.55 ns/elem
`PackedVector unpack (collect), n=65536`	29.1 µs	20.1 µs	3	0.44 ns/elem

All numbers from this machine/run; absolute values vary by host. Regenerate with `julia --project=benchmark benchmark/benchmarking.jl`.